Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #14

Surprise

TECHNICAL UNIVERSITY OF CLUJ-NAPOCA

Agenda

- **Surprise**

TECHNICAL UNIVERSITY OF CLUJ-NAPOCA

Surprise (Your questions)

- Quiz analysis – example of exercises
- Summary / Cheat Sheet

Quiz example

Given the compound term `n(c(A,B), v, m(1,[2,3]))`, which of the following terms it CANNOT unify with, and why?

Select one:

- a. N(c(a,B), v, m(1,[2,3])), because the name of the structure has to be constant
- b. n(A,B,C), because not all three arguments can be variable
- c. N(c(a,B), v, m(1,[H|T])), because the third arguments do not unify
- d. n(c(a,B), v, m(1,[2,B])), because the unifications involving B are in contradiction

Quiz example – contd.

Choose the right implementation of the `member` predicate to ensure a nondeterministic behavior:

Select one or more:

- [ ] a.
```
member(H,[H|_]).
member(H,[_|T]):-
 member(H,T).
```

- [ ] b.
```
member(H,[H|_]):-!.
member(H,[_|T]):-
 member(H,T).
```

- [ ] c.
```
member(H,[H|_]).
member(H,[_|T]):- !,
 member(H,T).
```

- [ ] d.
```
member(_,[]).
member(H,[H|_]).
member(H,[_|T]):-
 member(H,T).
```

Quiz example – contd.

The predicate below is assumed to remove **just the first** occurence of the first argument, from the list given in the second argument, to generate the list in the third argument.

To behave accordingly (as in the associated example), the code should be updated in the following way:

```
delete(X,[X|T],T).

delete(X,[H|T],[H|R]):-
     delete(X,T,R).

delete(_,[],[]).

?- delete(b,[a,b,c,b,d],R).

R= [a,c,b,d]? ;

no
```

Select one:

- a. Update the clause 1 as delete(X,[X|T],T):-!.
- b. Remove the clause delete(X,[X|T],T).
- c. Add the clause: delete(X,[X],[]).
- d. Remove the clause delete(_,[],[]).
- e. Swap the order of clauses 1 and 3.
- f. Update the clause 2 as delete(X,[H|T],[H|R]):-!,delete(X,T,R).
- g. Swap the order of clauses 1 and 2.
- h. Update the clause 2 as delete(X,[H|T], R):-!,delete(X,T,R).

Quiz example – contd.

The code below is assumed to collect in **IN**order the keys from a BST. Choose the best fit statement among the options:

```
tree_2_list_in(nil,_,_).
tree_2_list_in(t(Left,Key,Right),First,Last):-
 tree_2_list_in(Right,Int,Last),
 tree_2_list_in(Left,First,[Key|Int]).
```

Select one:

- a. The code is correct as it is if and only if the initial call sets the last argument to the empty list.
- b. To be correct, the order of the recursive calls must be swapped.
- c. The code is correct as it is.
- d. To be correct, in the first clause arguments 2 and 3 should share the same variable.

Quiz example – contd.

Given the edge representation of an undirected graph, the predicate `difference_graph` below generates the difference graph Gdiff of G1 and G2 (assume a predicate `is_edge` is present for each 3 graphs - G1 G2 and Gdiff):

```
difference_graph:-
      is_edge_1(X,Y),
      not(is_edge_2(X,Y)),
      not(is_edge_diff(X,Y)),
      assertz(edge_diff(X,Y)),
      fail.
difference_graph.

is_edge(X,Y):-
      edge(X,Y);
      edge(Y,X).
```

Choose the most specific correct answer from below:

Select one:

- a. The sequence is correct as both G1 and G2 are entirely scanned once.
- b. The sequence is incorrect, as it is missing a ! after `not(is_edge_2(X,Y))`.
- c. The sequence is correct as G1 is entirely scanned and G2 is scanned for each edge in G1.
- d. The sequence is incorrect, as it is missing a ! after `not(is_edge_diff(X,Y))`.

To find the value for "init“:
- you can think of the neutral element for that operation (ex. 0 for sum)
- or what is the result for the base case (ex. sum of all elements in [] is 0).

Summary / Cheat Sheet

% Template for complete list operations with backward recursion
```
pred([], "init").
pred([H|T], R):- pred(T, R1), R ⊖ R1 ⊕ H.
```

% Template for complete list operations with forward recursion
```
pred([], R, R).
pred([H|T], Acc, R):- Acc1 ⊖ Acc ⊕ H, pred(T, Acc1, R).

pred(L, R):- pred(L, "init", R).
```

% Template for complete list conditional operations
```
pred([], "init").
pred([H|T], R):- cond(...), !, pred(T, R1), R ⊖ R1 ⊕ H.
pred([H|T], R):- pred(T, R1), R ⊖ R1 ⊕.
```

% Template for deep list operations
```
pred([], "init").
pred([H|T], R):- atomic(H), !, pred(T, R1), R ⊖ R1 ⊕ H.
pred([H|T], R):- pred(H, R1), pred(T, R2), R ⊖ R1 ⊕ R2.
```

% Template for incomplete list operations
```
pred(L, "init"):-var(L),!.
pred([H|T], R):- pred(T, R1), R ⊖ R1 ⊕ H.
```

% Template for difference list (1st and 2nd arguments) operations
```
pred(S, E, "init"):- S==E,!.
pred([H|T], E, R):- pred(T, E, R1), R ⊖ R1 ⊕ H.
```

% Template for complete tree operations
```
pred(t(K,L,R), Result):-
    pred(L,NL),
    pred(R,NR),
    Result ⊖ NL ⊕ K ⊕ NR.
pred(nil, "init").
```

% Template for
% complete tree conditional operations
```
pred(t(K,L,R), Result):-
   cond(...), !,
   pred(L,NL),
   pred(R,NR),
   Result ⊖ NL ⊕ K ⊕ NR.
pred(t(K,L,R), Result):-
   pred(L,NL),
   pred(R,NR),
   Result ⊖ NL ⊕ NR.
pred(nil, "init").
```

% Template for incomplete tree operations [inorder]
```
pred(T, "init"):- var(T), !.
pred(t(K,L,R), Result):-
    pred(L,NL),
    pred(R,NR),
    Result ⊖ NL ⊕ K ⊕ NR.
```

Summary / Cheat Sheet

% Templates for dynamic operations (2 stage processes)

```
:-dynamic p/1.

p(1).
...
```

% Template for failure driven loop

```
pred(_):- failure_driven_loop.
pred(L):- collect(L).

failure_driven_loop:-
    p(X),          % / retract(p(X)),
    process(X),
    fail.
% no stopping condition, needs to fail to get to collect
```

```
process(X):- assert(q(X)).

collect([X|P]):-
  retract(visited_node(X)), !,
  collect(P).
collect([]).
```

% Template for retract recursion loop

```
pred(_):- retract_recursion.
pred(L):- collect(L).

retract_recursion:-
    retract(p(X)),                % / p(X), not(seen(X)), !, assert(seen(X)),
    process(X),
    retract_recursion.
% no stopping condition, needs to fail to get to collect
```

% Graphs

```
edge(1,2).
% ...

is_edge(X,Y):- edge(X,Y);edge(Y,X).
```

% Template for graph path operations

```
path(X, Y, Path):- path(X, Y, [X], Path).

path(Y, Y, _, [Y]).
path(X, Y, PPath, [X|Path]):-
    is_edge(X, Z),
    not(member(Z, PPath)),
    path(Z, Y, [Z|PPath], Path).


:- meta_predicate path(2,?,?,?).
path(G, X, Y, Path):- path(G, X, Y, [X], Path).

path(_,Y, Y, _, [Y]).
path(G, X, Y, PPath, [X|Path]):-
    call(G,X,Z),
    not(member(Z, PPath)),
    path(G, Z, Y, [Z|PPath], Path).
```